

tf.autodiff.ForwardAccumulator Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/autodiff/ForwardAccumulator

Computes Jacobian-vector products ("JVP"s) using forward-mode autodiff.

Function Signatures

```
tf.autodiff.ForwardAccumulator( primals, tangents )  
x = tf.constant([[2.0, 3.0], [1.0, 4.0]])  
targets = tf.constant([[1.], [-1.]])  
dense = tf.keras.layers.Dense(1)
```

Code Examples

```
tf.autodiff.ForwardAccumulator(  
    primals, tangents  
)
```

```
tf.autodiff.ForwardAccumulator(  
    primals, tangents  
)
```

```
x = tf.constant([[2.0, 3.0], [1.0, 4.0]])  
targets = tf.constant([[1.], [-1.]])  
dense = tf.keras.layers.Dense(1)  
dense.build([None, 2])  
with tf.autodiff.ForwardAccumulator(  
    primals=dense.kernel,  
    tangents=tf.constant([[1.], [0.]]) as acc:  
    loss = tf.reduce_sum((dense(x) - targets) ** 2.)  
    acc.jvp(loss)
```

```
x = tf.constant([[2.0, 3.0], [1.0, 4.0]])
```

```
targets = tf.constant([[1.], [-1.]])
```

```
dense = tf.keras.layers.Dense(1)
```

```
dense.build([None, 2])
```

```
with tf.autodiff.ForwardAccumulator(  
    primals=dense.kernel,  
    tangents=tf.constant([[1.], [0.]]) as acc:
```

Documentation

Computes Jacobian-vector products ("JVP"s) using forward-mode autodiff.

Compare to `tf.GradientTape` which computes vector-Jacobian products ("VJP"s) using reverse-mode autodiff (backprop). Reverse mode is more attractive when computing gradients of a scalar-valued function with respect to many inputs (e.g. a neural network with many parameters and a scalar loss). Forward mode works best on functions with many outputs and few inputs. Since it does not hold on to intermediate activations, it is much more memory efficient than backprop where it is applicable.

Consider a simple linear regression:

The example has two variables containing parameters, `dense.kernel` (2 parameters) and `dense.bias` (1 parameter). Considering the training data `x` as a constant, this means the Jacobian matrix for the function mapping from parameters to loss has one row and three columns.

With `forwardprop`, we specify a length-three vector in advance which multiplies the Jacobian. The `primals` constructor argument is the parameter (a `tf.Tensor` or `tf.Variable`) we're specifying a vector for, and the `tangents` argument is the "vector" in Jacobian-vector product. If our goal is to compute the entire Jacobian matrix, `forwardprop` computes one column at a time while `backprop` computes one row at a time. Since the Jacobian in the linear regression example has only one row, `backprop` requires fewer invocations:

Implicit in the `tape.gradient` call is a length-one vector which left-multiplies the Jacobian, a vector-Jacobian product.

ForwardAccumulator maintains JVPs corresponding primal tensors it is watching, derived from the original primals specified in the constructor. As soon as a primal tensor is deleted, ForwardAccumulator deletes the corresponding JVP.

`acc.jvp(x)` retrieves `acc`'s JVP corresponding to the primal tensor `x`. It does not perform any computation. `acc.jvp` calls can be repeated as long as `acc` is accessible, whether the context manager is active or not. New JVPs are only computed while the context manager is active.

Note that ForwardAccumulators are always applied in the order their context managers were entered, so inner accumulators will not see JVP computation from outer accumulators. Take higher-order JVPs from outer accumulators:

Reversing the collection in the last line to instead retrieve `inner.jvp(outer.jvp(primal_out))` will not work.

Strict nesting also applies to combinations of ForwardAccumulator and `tf.GradientTape`. More deeply nested GradientTape objects will ignore the products of outer ForwardAccumulator objects. This allows (for example) memory-efficient forward-over-backward computation of Hessian-vector products, where the inner GradientTape would otherwise hold on to all intermediate JVPs:

Raises

Methods

[View source](#)

Fetches the Jacobian-vector product computed for primals.

Note that this method performs no computation, and simply looks up a JVP that was already computed (unlike backprop using a `tf.GradientTape`, where the computation happens on the call to `tape.gradient`).

`__enter__`

[View source](#)

`__exit__`

[View source](#)